

Introduction to Node.js

- Node.js is a runtime environment.
- It allows JavaScript to run outside the browser.
- Used for building server-side applications.



What is Node.js?

Node.js is a runtime environment that lets you run JavaScript outside the browser — on your machine or a server. It's built on Google Chrome's V8 Engine, the same engine that powers Chrome's JavaScript execution.

Runtime Environment

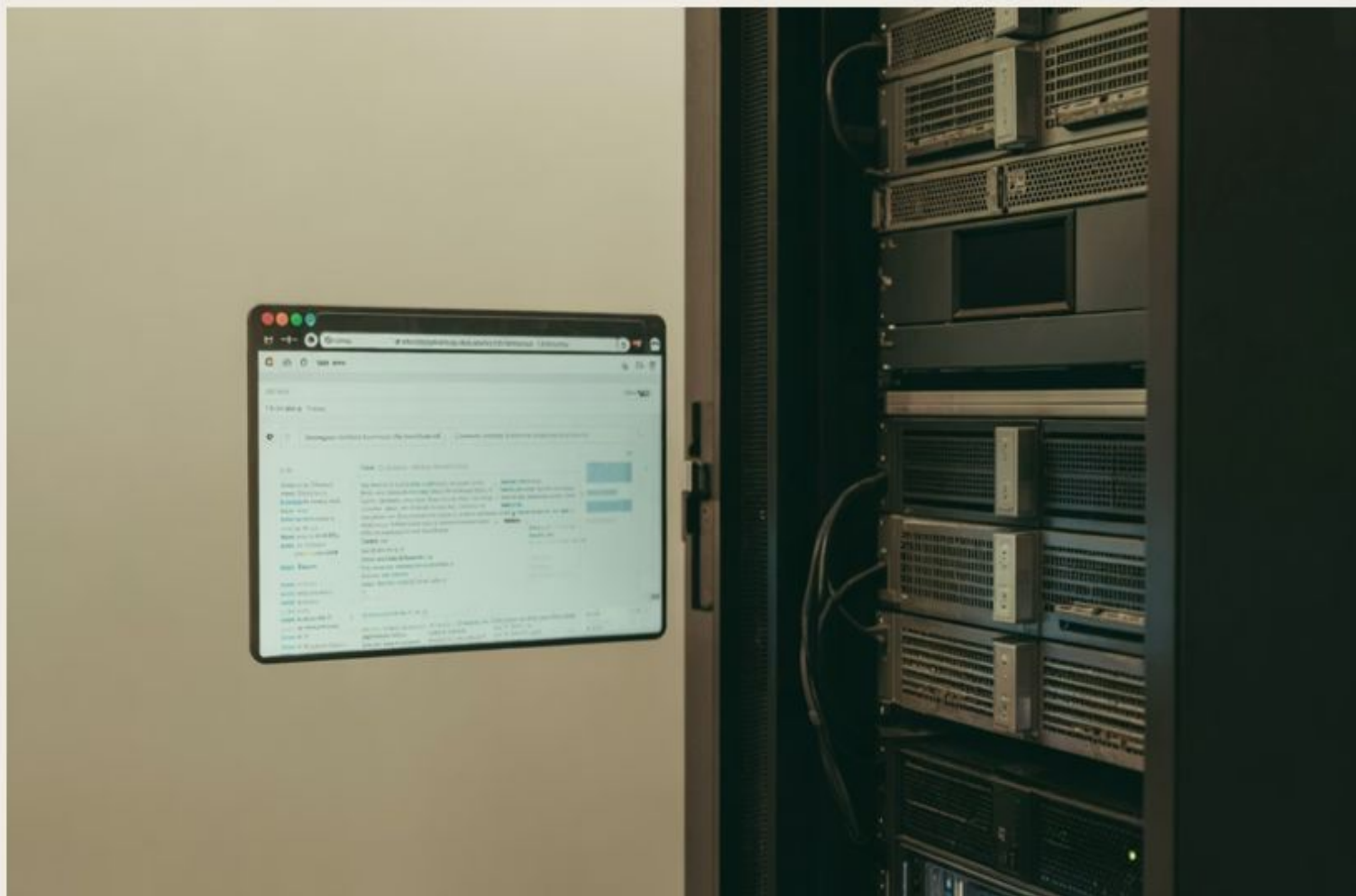
Executes JavaScript code
anywhere, not just in a browser

Server-Side

Used to build backends, APIs,
and web servers

Chrome V8 Engine

Delivers fast, high-performance JavaScript execution



THE PROBLEM

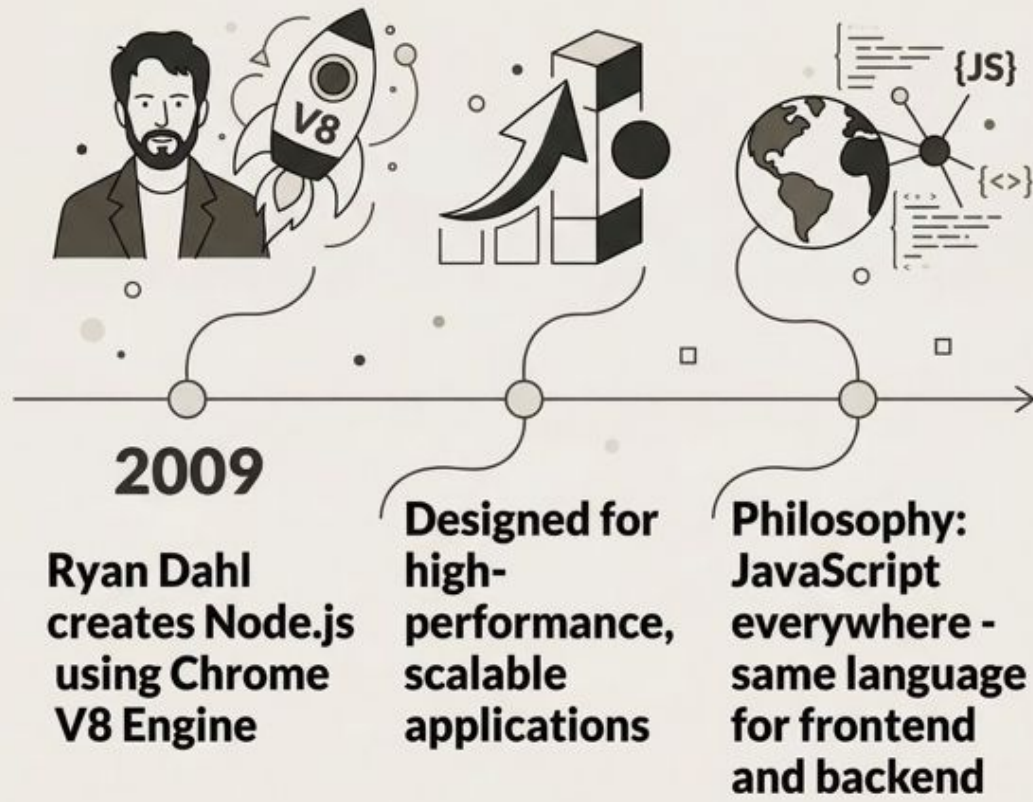
Why Node.js?

Before Node.js, JavaScript lived only in the browser. If you wanted server-side logic, you needed a different language — PHP, Python, Ruby, or Java.

Node.js changed that. It gave developers a way to use JavaScript on the server, unifying the stack and simplifying full-stack development.

📄 One language. Frontend and backend. That's the Node.js advantage.

History & Key Idea



The Vision

In 2009, developer Ryan Dahl introduced Node.js with a bold idea: use JavaScript — traditionally a browser-only language — to build fast, scalable server applications powered by the Chrome V8 Engine.

"JavaScript Everywhere"

With Node.js, the same language powers both the frontend (what users see) and the backend (server logic), streamlining development and reducing context-switching between languages.

Core Features of Node.js

Asynchronous & Non-Blocking

Tasks run without waiting. File reads, database queries, and network calls don't halt execution — boosting performance.

Event-Driven

Node.js responds to events (like user requests) using callbacks, making it ideal for real-time applications.

Fast Execution

Powered by Chrome's V8 Engine, Node.js compiles JavaScript to native machine code for high-speed performance.

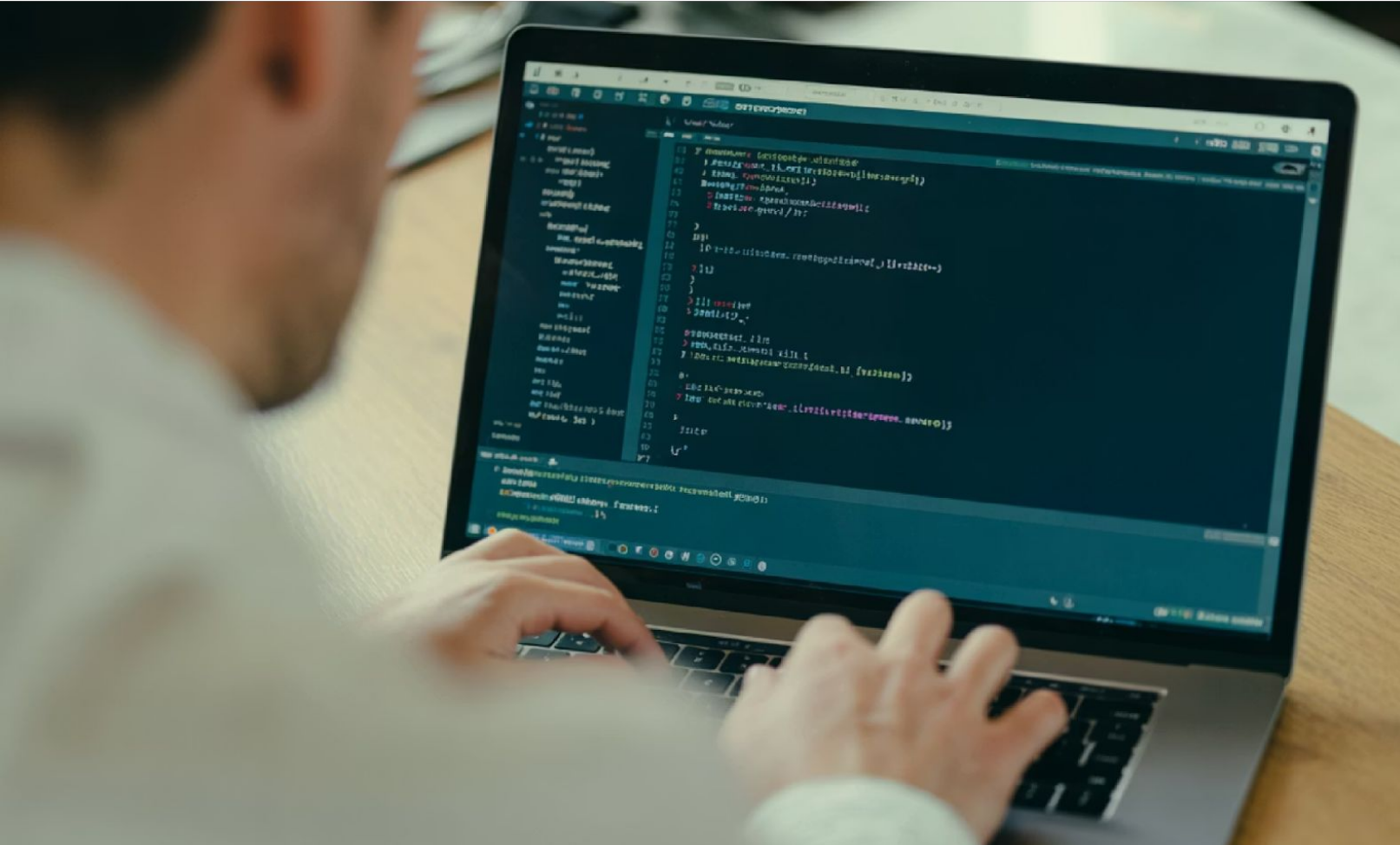
Scalable & Lightweight

Handles thousands of concurrent connections with minimal overhead — perfect for modern web applications.



Key Takeaway: Node.js uses non-blocking I/O — it never waits idle. While one task runs, it moves on to the next.

Asynchronous Programming Explained



How it Works

In asynchronous programming, tasks initiate and then continue running in the background without blocking the main program flow. The system is notified once a task completes, allowing it to handle other operations in the meantime.

This approach is crucial for performance, especially in server environments where applications need to handle many requests concurrently.

Example: When a Node.js application needs to read a file, it starts the read operation and immediately moves on to process other requests. Once the file is read, a callback function is executed.

Blocking vs. Non-Blocking Operations

1 Blocking Operations

When an operation is blocking, the program waits for that task to finish before moving on to the next line of code. This can lead to delays and reduced responsiveness, especially in applications that handle multiple users or I/O-intensive tasks.

Think of it like a single-lane road: only one car can pass at a time.

2 Non-Blocking Operations

Non-blocking operations allow the program to continue execution immediately after initiating a task, without waiting for it to complete. The program is notified later when the task is done.

Node.js primarily uses Non-Blocking I/O to ensure high performance and scalability.

Event Loop

- Core of Node.js.
- Handles asynchronous callbacks.
- Processes tasks in queue.

Modules in Node.js

- Reusable code blocks.
- Built-in modules like fs, http.
- Custom modules also possible.

File System Module

- Used to read/write files.
- Example: `fs.readFile()`.
- Works asynchronously.

HTTP Module

- Used to create web servers.
- Handles requests and responses.
- Core module.

NPM (Node Package Manager)

- Manages packages.
- Install libraries using `npm install`.
- Huge ecosystem.

Creating Server Example

- Basic server using http module.
- Listens on port.
- Responds to client.

Advantages

- Fast performance.
- Scalable.
- Large community.

Disadvantages

- Not ideal for CPU-heavy tasks.
- Callback complexity.
- Single-thread limitations.

Applications

- Web servers.
- APIs.
- Real-time apps like chat.
- Streaming services.

Node.js vs Browser JS

- Browser JS runs in browser.
- Node.js runs on server.
- Access to file system.

Frameworks

- Express.js.
- NestJS.
- Koa.
- Used for backend development.

JavaScript Engines

- Computers do not understand JavaScript
- A JavaScript engine takes JavaScript, and converts it into something it does understand - machine code

So...

- Node.js is written in C++
- At the heart of Node.js is the V8 engine
- The V8 engine converts our JS into machine code

Machine Code

JavaScript

C++

Assembly Language

Machine Code